

Mini Project Lesson: Basic Blog OR Contact Manager (PHP + MySQL)

1) Project Overview

What is a Basic Blog?

A small website where you can:

- create posts (title + content)
- show all posts (homepage)
- view one post (details page)
- edit/delete posts (admin area)

Real-world use cases

- personal blog, company announcements, school news, portfolio updates
-

What is a Contact Manager?

A small app to store people's info:

- name, phone, email, address, notes
- search and update contacts
- delete contacts safely

Real-world use cases

- small business client list, teacher's student contacts, personal address book
-

Features list (CRUD explained clearly)

CRUD = the 4 basic actions for database apps:

- **C — Create:** add new record (new post/contact)
- **R — Read:** show records (list + single view)

- **U — Update:** edit record
 - **D — Delete:** remove record
-

Which option is better for beginners, and why?

✓ **Contact Manager is usually better for beginners** because:

- Data is more “structured” (name/phone/email) → easier to validate
- You naturally learn forms and validation
- “Post content” in blog is long text → beginners often struggle with formatting + XSS

✓ **BUT** blog is great if students want to feel “website-like” quickly.

Instructor suggestion: Teach **Contact Manager** first, then say:
“Blog is the same CRUD with different fields.”

In this lesson, we’ll build **Contact Manager**, and I’ll note the blog changes where needed.

2) Concept Map (Graphical Explanation)

A) Browser → PHP → MySQL → Response

```
[Browser]
|
| 1) Request (GET/POST)
v
[PHP Script]
|
| 2) Validate input + build query (prepared statement)
v
[MySQL Database]
|
| 3) Return rows / success
v
[PHP Script]
|
| 4) Generate HTML response
v
[Browser shows page]
```

B) Folder structure map

```
project/
├─ public/          (Pages user can open)
```

└─ admin/	(Management pages: create/edit/delete)
└─ config/	(Database settings, constants)
└─ includes/	(Reusable parts: header/footer/functions)
└─ assets/	(CSS, images, JS)

C) CRUD data flow (example: “Create Contact”)

```

User fills form
  |
  v
POST /admin/create.php
  |
  v
Validate data (server-side)
  |
  v
INSERT into database (PDO prepared statement)
  |
  v
Redirect to /public/index.php?success=1

```

3) Project Folder Structure

Recommended structure (simple but real-world)

```

contact-manager/
├─ admin/
│   ├── create.php
│   ├── edit.php
│   ├── delete.php
│   └── _protect.php          (optional if login added later)
├─ config/
│   └─ database.php
├─ includes/
│   ├── header.php
│   ├── footer.php
│   └─ functions.php
├─ public/
│   ├── index.php            (list contacts)
│   └─ view.php              (single contact)
├─ assets/
│   └─ style.css
└─ .env (optional later)

```

Why this structure matters (simple analogy)

Think of your project like a **school**:

- public/ = classrooms (students can enter)
- admin/ = teacher office (manage records)

- `config/` = school rules (DB settings)
- `includes/` = shared resources (timetable, forms, headers)
- `assets/` = uniforms/books (CSS/images)

✓ This prevents messy “everything in one file” projects.

4) 📊 Database Design (MySQL)

Step-by-step: create database + table

Run in phpMyAdmin or MySQL client:

```
CREATE DATABASE contact_manager CHARACTER SET utf8mb4 COLLATE
utf8mb4_unicode_ci;
USE contact_manager;
```

✗ Bad table design (what beginners often do)

```
CREATE TABLE contacts (
  id VARCHAR(100),
  name TEXT,
  phone TEXT,
  email TEXT,
  address TEXT,
  created TEXT
);
```

Why it's bad

- `id` should be numeric + auto generated (faster, simpler)
 - `TEXT` for everything wastes space and hurts indexing
 - `created` should be a real date/time type, not `TEXT`
-

✓ Optimized design (recommended)

```
CREATE TABLE contacts (
  id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  full_name VARCHAR(100) NOT NULL,
  phone VARCHAR(30) NOT NULL,
  email VARCHAR(120) NULL,
  address VARCHAR(255) NULL,
  notes TEXT NULL,
```

```
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
updated_at TIMESTAMP NULL DEFAULT NULL ON UPDATE CURRENT_TIMESTAMP  
) ENGINE=InnoDB;
```

Explain columns + data types (student-friendly)

- `id INT AUTO_INCREMENT` → database assigns 1,2,3... automatically
 - `VARCHAR (100)` → short text (name/email) with a max length
 - `TEXT` → long text (notes/blog content) with flexible length
 - `TIMESTAMP` → date/time automatically stored by database
-

Explain key concepts

PRIMARY KEY = the “ID card number” of each row (unique)

AUTO_INCREMENT = “system gives the next ID”

VARCHAR vs TEXT

- `VARCHAR` = short text, faster for indexing/search
 - `TEXT` = big text, not ideal for indexing, but good for notes/content
-

✓ Blog version table (if you choose blog instead)

```
CREATE TABLE posts (  
  id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,  
  title VARCHAR(180) NOT NULL,  
  body TEXT NOT NULL,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

5) Database Connection

A) ✗ Weak practice (mysqli, inline creds, no error handling)

```
<?php  
$conn = mysqli_connect("localhost", "root", "", "contact_manager");  
// If it fails, the page might just break silently...  
?>
```

Why it's bad

- No proper error messages
- Mixing connection code everywhere

- Hard to maintain

B) Best practice (PDO, reusable file, good errors)

Create: config/database.php

```
<?php
// config/database.php

$host = 'localhost';
$db   = 'contact_manager';
$user = 'root';
$pass = '';
$charset = 'utf8mb4';

$dsn = "mysql:host=$host;dbname=$db;charset=$charset";

$options = [
    PDO::ATTR_ERRMODE            => PDO::ERRMODE_EXCEPTION, // show real errors
    PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,        // arrays by column
    name
    PDO::ATTR_EMULATE_PREPARES   => false,                    // real prepares
];

try {
    $pdo = new PDO($dsn, $user, $pass, $options);
} catch (PDOException $e) {
    // Simple message for students (don't show secrets in real production)
    die("Database connection failed: " . $e->getMessage());
}
```

Why it's better

- One clean connection file
- Strong error handling
- Works great with prepared statements (security)

6) CREATE (Insert Data)

A) Create the form (admin/create.php)

Explanation first:

The form sends data using **POST** because we are **changing data**.

```

<?php
require_once __DIR__ . '/../config/database.php';
require_once __DIR__ . '/../includes/functions.php';
include __DIR__ . '/../includes/header.php';
?>

<h2>Add New Contact</h2>

<form method="POST" action="create.php">
  <label>Full Name *</label><br>
  <input type="text" name="full_name" required><br><br>

  <label>Phone *</label><br>
  <input type="text" name="phone" required><br><br>

  <label>Email</label><br>
  <input type="email" name="email"><br><br>

  <label>Address</label><br>
  <input type="text" name="address"><br><br>

  <label>Notes</label><br>
  <textarea name="notes"></textarea><br><br>

  <button type="submit" name="submit">Save Contact</button>
</form>

<?php include __DIR__ . '/../includes/footer.php'; ?>

```

B) PHP handling + validation + insert (same file, below form OR top)

Good classroom style: handle POST at top of same file.

Add **above** the HTML in admin/create.php:

✗ *Weak code*

```

// BAD: directly trusting user input (SQL injection risk)
$sql = "INSERT INTO contacts (full_name, phone) VALUES ('$_POST[full_name]',
'$_POST[phone]')";
$pdo->query($sql);

```

Why it's bad

- Hacker can inject SQL
 - No validation
 - Breaks easily with quotes like O'Brien
-

Best practice code (prepared statements + validation)

```
<?php
require_once __DIR__ . '/../config/database.php';
require_once __DIR__ . '/../includes/functions.php';

$errors = [];

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    // 1) Get & clean inputs
    $full_name = trim($_POST['full_name'] ?? '');
    $phone      = trim($_POST['phone'] ?? '');
    $email      = trim($_POST['email'] ?? '');
    $address    = trim($_POST['address'] ?? '');
    $notes      = trim($_POST['notes'] ?? '');

    // 2) Basic validation
    if ($full_name === '') $errors[] = "Full name is required.";
    if ($phone === '') $errors[] = "Phone is required.";

    // 3) Improved validation
    if ($email !== '' && !filter_var($email, FILTER_VALIDATE_EMAIL)) {
        $errors[] = "Email format is not valid.";
    }

    // 4) Insert if no errors
    if (empty($errors)) {
        $stmt = $pdo->prepare("
            INSERT INTO contacts (full_name, phone, email, address, notes)
            VALUES (:full_name, :phone, :email, :address, :notes)
        ");

        $stmt->execute([
            ':full_name' => $full_name,
            ':phone'      => $phone,
            ':email'      => $email !== '' ? $email : null,
            ':address'    => $address !== '' ? $address : null,
            ':notes'      => $notes !== '' ? $notes : null,
        ]);

        // PRG pattern: Post/Redirect/Get (prevents duplicate insert on refresh)
        header("Location: ../public/index.php?created=1");
        exit;
    }
}
?>
```

Now show errors (in the HTML part):

```
<?php if (!empty($errors)): ?>
<div style="color:red;">
<ul>
    <?php foreach ($errors as $err): ?>
        <li><?= e($err) ?></li>
    <?php endforeach; ?>
</ul>
```



```
</div>
<?php endif; ?>
```

Common beginner mistakes (Create)

- Refreshing after POST → inserts duplicate (solve with redirect)
 - Using GET for insert (don't)
 - Not trimming inputs (spaces become "data")
 - Not validating email
 - Not using prepared statements
-

7) READ (Display Data)

A) List all contacts (public/index.php)

Goal: fetch from database and loop to display.

Code

```
<?php
require_once __DIR__ . '/../config/database.php';
require_once __DIR__ . '/../includes/functions.php';
include __DIR__ . '/../includes/header.php';

$stmt = $pdo->query("SELECT id, full_name, phone, email, created_at FROM
contacts ORDER BY id DESC");
$contacts = $stmt->fetchAll();
?>

<h2>All Contacts</h2>

<p>
  <a href="../admin/create.php">+ Add Contact</a>
</p>

<?php if (isset($_GET['created'])): ?>
  <p style="color:green;"> Contact created successfully!</p>
<?php endif; ?>

<?php if (empty($contacts)): ?>
  <p>No contacts found. Add your first contact! 😊</p>
<?php else: ?>
  <table border="1" cellpadding="8">
    <tr>
      <th>Name</th>
      <th>Phone</th>
      <th>Email</th>
      <th>Action</th>
```

```

</tr>

<?php foreach ($contacts as $c): ?>
    <tr>
        <td><?= e($c['full_name']) ?></td>
        <td><?= e($c['phone']) ?></td>
        <td><?= e($c['email'] ?? '-') ?></td>
        <td>
            <a href="view.php?id=<?= (int)$c['id'] ?>">View</a> |
            <a href="../admin/edit.php?id=<?= (int)$c['id'] ?>">Edit</a> |
            <a href="../admin/delete.php?id=<?= (int)$c['id'] ?>">Delete</a>
        </td>
    </tr>
<?php endforeach; ?>
</table>
<?php endif; ?>

<?php include __DIR__ . '/../includes/footer.php'; ?>

```

B) Single contact view (public/view.php)

```

<?php
require_once __DIR__ . '/../config/database.php';
require_once __DIR__ . '/../includes/functions.php';
include __DIR__ . '/../includes/header.php';

$id = (int)($_GET['id'] ?? 0);

$stmt = $pdo->prepare("SELECT * FROM contacts WHERE id = :id");
$stmt->execute([':id' => $id]);
$contact = $stmt->fetch();

if (!$contact) {
    echo "<p>Contact not found.</p>";
    include __DIR__ . '/../includes/footer.php';
    exit;
}
?>

<h2><?= e($contact['full_name']) ?></h2>
<p><b>Phone:</b> <?= e($contact['phone']) ?></p>
<p><b>Email:</b> <?= e($contact['email'] ?? '-') ?></p>
<p><b>Address:</b> <?= e($contact['address'] ?? '-') ?></p>
<p><b>Notes:</b><br><?= nl2br(e($contact['notes'] ?? '')) ?></p>

<p><a href="index.php">⬅ Back</a></p>

<?php include __DIR__ . '/../includes/footer.php'; ?>

```

Security explanation (XSS)

If a user saves:

```
<script>alert('Hacked')</script>
```

...and you output it directly, the browser runs it. That's **XSS**.

✓ Fix: always escape output with `e()` function (below).

8) UPDATE (Edit Data)

Explain URL parameters clearly

When you click:

```
edit.php?id=5
```

- `id` is a **URL parameter**
 - PHP reads it from `$_GET['id']`
-

A) Create edit page (admin/edit.php)

Steps:

1. Get `id` from URL
2. Fetch existing data
3. Show form pre-filled
4. On POST, update row

✓ Code

```
<?php
require_once __DIR__ . '/../config/database.php';
require_once __DIR__ . '/../includes/functions.php';

$id = (int)($_GET['id'] ?? 0);
$errors = [];

// 1) Fetch existing contact
$stmt = $pdo->prepare("SELECT * FROM contacts WHERE id = :id");
$stmt->execute([':id' => $id]);
$contact = $stmt->fetch();

if (!$contact) {
    die("Contact not found.");
}

// 2) Handle update
```

```

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $full_name = trim($_POST['full_name'] ?? '');
    $phone      = trim($_POST['phone'] ?? '');
    $email       = trim($_POST['email'] ?? '');
    $address     = trim($_POST['address'] ?? '');
    $notes       = trim($_POST['notes'] ?? '');

    if ($full_name === '') $errors[] = "Full name is required.";
    if ($phone === '') $errors[] = "Phone is required.";
    if ($email !== '' && !filter_var($email, FILTER_VALIDATE_EMAIL)) {
        $errors[] = "Email format is not valid.";
    }

    if (empty($errors)) {
        $stmt = $pdo->prepare("
            UPDATE contacts
            SET full_name = :full_name,
                phone = :phone,
                email = :email,
                address = :address,
                notes = :notes
            WHERE id = :id
        ");

        $stmt->execute([
            ':full_name' => $full_name,
            ':phone'      => $phone,
            ':email'      => $email !== '' ? $email : null,
            ':address'    => $address !== '' ? $address : null,
            ':notes'      => $notes !== '' ? $notes : null,
            ':id'         => $id,
        ]);

        header("Location: ../public/view.php?id=$id&updated=1");
        exit;
    }
}

include __DIR__ . '/../includes/header.php';
?>

<h2>Edit Contact</h2>

<?php if (!empty($errors)): ?>
    <div style="color:red;">
        <ul>
            <?php foreach ($errors as $err): ?>
                <li><?= e($err) ?></li>
            <?php endforeach; ?>
        </ul>
    </div>
<?php endif; ?>

<form method="POST">
    <label>Full Name *</label><br>
    <input type="text" name="full_name" value="<?= e($contact['full_name']) ?>"
required><br><br>

```

```
<label>Phone *</label><br>
<input type="text" name="phone" value="<?= e($contact['phone']) ?>"
required><br><br>

<label>Email</label><br>
<input type="email" name="email" value="<?= e($contact['email'] ?? '')
?>"><br><br>

<label>Address</label><br>
<input type="text" name="address" value="<?= e($contact['address'] ?? '')
?>"><br><br>

<label>Notes</label><br>
<textarea name="notes"><?= e($contact['notes'] ?? '') ?></textarea><br><br>

<button type="submit">Update</button>
</form>

<?php include __DIR__ . '/../includes/footer.php'; ?>
```

9) DELETE (Remove Data)

Why delete is dangerous (classroom explanation)

Delete is like **throwing away a file permanently**.
One wrong click = data gone.

So we do:

- confirmation step
- use POST (not GET) for final delete

✗ Weak delete (danger!)

```
// BAD: clicking a link like delete.php?id=5 deletes instantly using GET
$pdo->query("DELETE FROM contacts WHERE id = " . $_GET['id']);
```

Why it's bad

- Anyone can trigger it by sending a link
 - No confirmation
 - SQL injection risk
-

✓ Safe delete with confirmation (admin/delete.php)

```
<?php
require_once __DIR__ . '/../config/database.php';
require_once __DIR__ . '/../includes/functions.php';

$id = (int)($_GET['id'] ?? 0);

// Fetch contact to show name in confirmation
$stmt = $pdo->prepare("SELECT id, full_name FROM contacts WHERE id = :id");
$stmt->execute([':id' => $id]);
$contact = $stmt->fetch();

if (!$contact) {
    die("Contact not found.");
}

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    // Final delete happens only on POST
    $stmt = $pdo->prepare("DELETE FROM contacts WHERE id = :id");
    $stmt->execute([':id' => $id]);

    header("Location: ../public/index.php?deleted=1");
    exit;
}

include __DIR__ . '/../includes/header.php';
?>

<h2>Delete Contact</h2>
<p>Are you sure you want to delete <b><?= e($contact['full_name'])
?></b>?</p>

<form method="POST">
    <button type="submit">Yes, Delete</button>
    <a href="../public/view.php?id=<?= (int)$contact['id'] ?>">Cancel</a>
</form>

<?php include __DIR__ . '/../includes/footer.php'; ?>
```

10) Security Basics (with attack vs protection)

A) SQL Injection

Attack example: user enters in “name”:

```
anything'); DROP TABLE contacts; --
```

✗ Vulnerable code

```
$sql = "INSERT INTO contacts (full_name) VALUES ('$full_name')";
$pdo->query($sql);
```

✓ Protected code (prepared statement)

```
$stmt = $pdo->prepare("INSERT INTO contacts (full_name) VALUES (:full_name)");  
$stmt->execute([':full_name' => $full_name]);
```

B) XSS (Cross-Site Scripting)

✗ Vulnerable output

```
echo $contact['notes']; // runs HTML/JS
```

✓ Safe output

```
echo e($contact['notes']);
```

C) Password hashing (if login is used later)

✗ Wrong

```
$password = md5($_POST['password']);
```

✓ Correct

```
$hash = password_hash($_POST['password'], PASSWORD_DEFAULT);
```

```
// verify later:  
password_verify($passwordInput, $hash);
```

11) ⚠ Weak Code vs Best Practice (MANDATORY SUMMARY)

Below is a quick “compare list” for every major topic we used:

Database connection

- ✗ Inline connection in every file
- ✓ One config/database.php + PDO + exceptions

Insert/Update/Delete queries

- ✗ String concatenation SQL
- ✓ Prepared statements

Forms

- ✗ Trusting \$_POST directly

- ☒ `trim()`, required checks, `filter_var()`

Output

- ☒ `echo $data`
- ☒ `echo e($data)` to prevent XSS

Delete

- ☒ Delete on GET link click
- ☒ Confirmation page + delete only on POST

Refresh after POST

- ☒ Duplicate inserts
- ☒ PRG pattern: redirect after success

12) ☐ Debugging & Common Errors

A) Database connection errors

Symptoms: blank page / “connection failed”

Fix checklist:

- correct database name
- correct username/password
- MySQL running?
- try echo test: `echo "Connected!"`;

B) Undefined index (common!)

Example:

```
$name = $_POST['full_name']; // error if not submitted
```

☒ Fix:

```
$name = $_POST['full_name'] ?? '';
```

C) “Header already sent”

Cause: output before `header()`, like:

- spaces before `<?php`
- `echo` before redirect

✓ Fix:

- keep PHP redirect logic at top
 - no `echo` before header
 - use `exit;` after header
-

D) How to read error messages (student method)

Tell students:

1. Read the last line (most important)
 2. Find the file + line number
 3. Open that line and check variable names / semicolons
-

13) □ Mini Tasks for Students (after each section)

After Folder Structure

- Task: Add a new folder `partials/` and move `header.php/footer.php` there.

After Database Design

- Task: Add a new column `birthday DATE NULL`
- Task: Explain why `DATE` is better than `VARCHAR` for birthdays

After Connection

- Task: Change database name wrong on purpose, read the error, then fix it.

After CREATE

- Task: Add validation: phone must be at least 6 characters
- Task: Break it: remove `trim()` and see what happens with spaces

After READ

- Task: Add a “Search by name” form (GET) and filter query with LIKE (prepared)

After UPDATE

- Task: Add `updated_at` display on view page

After DELETE

- Task: Add “soft delete” idea (extra): add `is_deleted TINYINT(1)` and filter records

After Security

- Task: Try saving `<b>Hello</b>` in notes and confirm it doesn’t render as HTML (because of escaping)

After Debugging

- Task: Create an “Undefined variable” error intentionally and solve it

14) 🧠 Interview & Viva Questions (with simple answers)

1. **What is CRUD?**
Create, Read, Update, Delete—basic database operations.
2. **Why use POST for create/update/delete?**
Because it changes data; GET is for reading.
3. **What is a primary key?**
A unique identifier for each row.
4. **Why AUTO_INCREMENT?**
Database automatically generates the next ID.
5. **What is SQL Injection?**
When attacker manipulates SQL using input fields.
6. **How do prepared statements help?**
They separate SQL from data, blocking injection.
7. **What is XSS?**
Injecting scripts into pages via saved input/output.
8. **How do you prevent XSS in PHP?**
Escape output (e.g., `htmlspecialchars`).
9. **VARCHAR vs TEXT?**
VARCHAR for short text; TEXT for long content.
10. **Why redirect after insert?**
Prevents duplicate inserts on refresh.

11. **What does `$_GET` contain?**
Data from URL query string.
 12. **What does `$_POST` contain?**
Data submitted by form using POST.
 13. **Why validate on server side if HTML required exists?**
Because HTML checks can be bypassed.
 14. **What does `exit;` do after header redirect?**
Stops script execution so no extra output runs.
 15. **What is `password_hash()`?**
Safe way to store passwords using modern hashing.
-

15) Final Project Checklist (Before Submission)

☒ Structure

- ☐ Separate folders (public/admin/config/includes/assets)
- ☐ Reusable header/footer

☒ Database

- ☐ Proper types (INT PK, VARCHAR, TEXT, TIMESTAMP)
- ☐ No “everything TEXT”

☒ CRUD

- ☐ Create works + validation
- ☐ List page works + empty state message
- ☐ Single view works
- ☐ Edit works (prefilled)
- ☐ Delete confirmation + delete on POST only

☒ Security

- ☐ Prepared statements everywhere
- ☐ Escape output to prevent XSS
- ☐ No direct SQL with user input

☑ Quality

- ☐ Redirect after successful POST
- ☐ Clean error messages for users
- ☐ Comments in code

Required Helper File: **includes/functions.php**

Create this file (we used it above):

```
<?php
// includes/functions.php

function e(string $value): string {
    // Escape output to prevent XSS
    return htmlspecialchars($value, ENT_QUOTES, 'UTF-8');
}
```

☑ Required Basic Layout Files

includes/header.php

```
<!DOCTYPE html>
<html>
<head>
    <meta charset="UTF-8">
    <title>Contact Manager</title>
    <link rel="stylesheet" href="../../assets/style.css">
</head>
<body>
    <div style="max-width:900px;margin:20px auto;font-family:Arial;">
```

includes/footer.php

```
    </div>
</body>
</html>
```